

Ex. No: 3 System Calls: Fork, Exit, Getpid, Wait, Close

1. fork()

C PROGRAM

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
int main()
```

```
{
```

```
pid_t pid;
```

```
pid = fork();
```

```
if(pid < 0)
```

```
{
```

```
printf("Fork Failed\n");
```

```
exit(1);
```

```
}
```

```
else if(pid == 0)
```

```
{
```

```
printf("\nCHILD PROCESS");
```

```
printf("\nChild PID : %d", getpid());
```

```
printf("\nParent PID : %d\n", getppid());
```

```
exit(0);
```

```
}
```

```
else
```

```
{
```

```
wait(NULL);

printf("\nPARENT PROCESS");

printf("\nParent PID : %d", getpid());

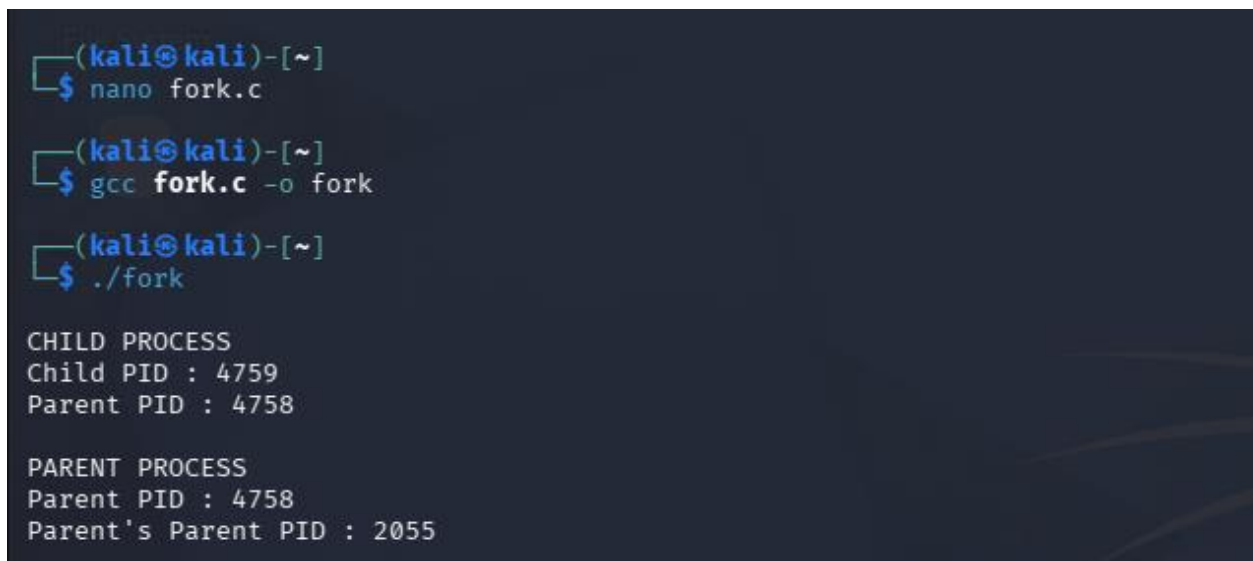
printf("\nParent's Parent PID : %d\n", getppid());

}

return 0;

}
```

Output:



```
(kali@kali)-[~]
$ nano fork.c

(kali@kali)-[~]
$ gcc fork.c -o fork

(kali@kali)-[~]
$ ./fork

CHILD PROCESS
Child PID : 4759
Parent PID : 4758

PARENT PROCESS
Parent PID : 4758
Parent's Parent PID : 2055
```

Shell Script:

```
#!/bin/bash

echo "Parent Process ID : $$"

(
echo "Child Process ID : $$"
echo "Parent Process ID : $PPID"
exit 0
) &
```

wait

echo "Child Process Completed"

Output:

```
(kali㉿kali)-[~]  
$ nano lab3-1.sh  
  
(kali㉿kali)-[~]  
$ chmod +x lab3-1.sh  
  
(kali㉿kali)-[~]  
$ bash lab3-1.sh  
Parent Process ID : 38282  
Child Process ID : 38282  
Parent Process ID : 19424  
Child Process Completed
```

2. WAIT()

C PROGRAM

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
int main() {
```

```
    pid_t pid;
```

```
    pid = fork();
```

```
    if(pid == 0)    {
```

```
        printf("Child Process Running
```

```
        \n");
```

```
        sleep(5);
```

```
        printf("Child Process Completed
```

```
        \n");    }
```

```
    else{
```

```
        wait(NULL);
```

```
printf("Parent Resumes Execution\n");  
}  
return 0; }
```

Output:

```
(kali㉿kali)-[~]  
$ nano wait.c  
  
(kali㉿kali)-[~]  
$ gcc wait.c -o wait  
  
(kali㉿kali)-[~]  
$ ./wait  
Child Process Running  
Child Process Completed  
Parent Resumes Execution
```

SHELL SCRIPT

```
#!/bin/bash (  
echo "Child Process Running"  
sleep 5  
echo "Child Process Completed"  
) &  
wait  
echo "Parent Resumes Execution"
```

Output:

```
(kali㉿kali)-[~]  
$ nano lab3-2.sh  
  
(kali㉿kali)-[~]  
$ chmod +x lab3-2.sh  
  
(kali㉿kali)-[~]  
$ bash lab3-2.sh  
Child Process Running  
Child Process Completed
```

3: CLOSE()

C PROGRAM

```
#include <stdio.h>
```

```
#include <fcntl.h>
```

```
#include <unistd.h>
```

```
int main()
```

```
{
```

```
int fd;
```

```
fd = open("sample.txt", O_RDONLY);
```

```
if(fd < 0)
```

```
{
```

```
printf("File Opening Failed\n");
```

```
return 1;
```

```
}
```

```
printf("File Opened Successfully\n");
```

```
close(fd);
```

```
printf("File Closed Successfully\n");
```

```
return 0;
```

```
}
```

Output:

```
(kali㉿kali)-[~]  
$ nano close.c  
  
(kali㉿kali)-[~]  
$ gcc close.c -o close  
  
(kali㉿kali)-[~]  
$ ./close  
File Opening Failed
```

SHELL SCRIPT

Shell scripting does not provide a direct `close()` system call. File descriptors can be opened and closed

as follows:

```
#!/bin/bash
```

```
exec 3< sample.txt
```

```
echo "File Opened Successfully"
```

```
exec 3<&-
```

```
echo "File Closed Successfully"
```

Output:

```
(kali㉿kali)-[~]  
  
(kali㉿kali)-[~]  
$ chmod +x lab3-3.sh  
  
(kali㉿kali)-[~]  
$ bash lab3-3.sh  
lab3-3.sh: line 2: sample.txt: No such file or directory  
File Opened Successfully  
File Closed Successfully
```